

CertiLedger

The Blockchain-Based Certificate Verification System

Mentor: Prof. Rikita Choksi

Version 1.0

TEAM MEMBERS

Vatsal Bharkhada (22CE006)

Isha Jivani (22CE041)

Rohan Rohit (D23CE162)

Introduction & Project Scope

Purpose

CertiLedger addresses the issue of forged academic documents by providing a secure, reliable, and tamper-resistant platform. It stores cryptographic hash values of certificates on a blockchain while maintaining metadata in a secure database.

Scope

- ✓ **Issuance:** Generation of digital certificates by authorized institutions.
- ✓ **Storage:** Secure storage of data with blockchain hashes for integrity.
- ✓ **Verification:** Public verification capability without authentication.



System Architecture & Tech Stack



Frontend

Web-based interface built with React.js. Designed for Admins, Students, and Public Verifiers.

Technology:
React.js



Backend & Database

Application server handling logic and API requests, connected to a relational database for user and certificate metadata.

Technology: PHP +
MySQL



Blockchain Layer

Immutable storage for cryptographic hashes. Ensures that once a certificate is issued, its validity cannot be tampered with.

Technology: Ethereum
(Ganache)

Core System Features

Certificate Issuance

Authorized admins can issue certificates. The system generates a unique ID and a cryptographic hash which is registered on the blockchain.

Access Control (RBAC)

Strict role management (Admin, Student, Public) ensures security. Only authorized personnel can perform write operations on the ledger.

Public Verification

Allows external users (e.g., employers) to verify authenticity by uploading a file or entering an ID. The system compares the live hash against the blockchain.

Revocation Management

Admins can revoke invalid certificates. The revocation status is permanently recorded on the blockchain to prevent misuse.

System Workflow

1

Admin Login

University authority authenticates
into the system securely.

2

Issuance

Admin inputs details. System
generates metadata and unique
hash.

3

Registration

Cryptographic hash is written to
the Ethereum blockchain;
Metadata to SQL.

4

Verification

Public user submits ID. System
compares current hash with
blockchain record.

Non-Functional Requirements



Security & Integrity

- ✓ Data integrity ensured via blockchain hashing.
- ✓ Secure authentication for admin operations.
- ✓ Public verification does not expose sensitive student PII.



Performance

- ✓ Asynchronous handling of blockchain transactions to prevent UI lag.
- ✓ Support for multiple concurrent users.
- ✓ Efficient verification response times.



Quality Attributes

- ✓ **Reliability:** Consistent verification results.
- ✓ **Maintainability:** Modular architecture.
- ✓ **Portability:** Deployable on standard web servers.

Thank You
